

TC2037 Evidence 1: Implementation of Lexical Analysis

Moritz Yulian Trieu: A01831900

14/03/2026

1 Introduction

This evidence looks at finding a finite automaton to accept a provided regular language as also described in [Frank Stakano, 2024]. The resulting automaton can then be either modeled using Prolog or REGEX as for example done in [Geeks-For-Geeks, 2026]. The language chosen for this evidence possesses the alphabet $\Sigma = \{0, 1, 2\}$, needs to end in either "111" or "122" to transition into an accepting state and prohibits the occurrence of "211".

2 Automaton

As [Hopcroft et al., 2007] mention, (deterministic) finite automaton are useful models for different tasks, including lexical analysis. When interpreting finite automata as transition graphs, this can be easily visualized. A string can be broken down into each letter, which can be used to follow a graph into a specific node/ state. In [Hopcroft et al., 2007] a deterministic finite automaton is defined as a quintuple $A = (Q, \Sigma, \delta, q_0, F)$ where

- Q defines a finite set of states,
- Σ a finite alphabet,
- δ a transition function,
- q_0 the initial state and
- F the accepting states.

As indicated in [Frank Stakano, 2024], an automaton of this kind parses a string by transitioning to different states of the automaton according to the transition function given the characters of the input string. In fig. 1 a basic example of passing a binary string is given. An input gets accepted if, after passing the string, it ends up in an accepting state, otherwise it is rejected.

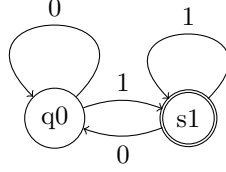


Figure 1: In this very basic example, $\Sigma = \{0, 1\}$ where a string has to end in "1". If given a string "1001", each character of the string will be consumed and will transition the automaton in a new state according to the edge labels. Since the last digit is "1", the automaton will end up in an accepting state.

In [Hopcroft et al., 2007] the existence of non-deterministic finite automaton is mentioned, But as further explained, a non-deterministic model has to be converted to a deterministic model, in order to be implementable. Due to the small size and lower complexity of the language mentioned in section 1, a deterministic model approach was possible and therefore chosen, to reduce complexity.

2.1 Modeling the language

Each string should start out at the same entry state E , so $q0 = E$. From there one can derive the different conditions path, the automaton needs to recognize. In fig. 2 a first model is depicted that encodes all paths that are important for acceptance and rejection and therefore also determines all required states. The path $s1e-s1f$ shows the accepted ending sequence "111", $s1e-s2f$ models the accepting end string "122" and $s3e-s3f$ show the rejected sequence "211". With this F is defined as $F = \{s1f, s2f\}$. This graph is incomplete, as each state needs to have a transition for each symbol in the alphabet. In fig. 3, the full model is shown with all connections. Figure 3 further implies Q and δ . Important to observe is, that each occurrence of "0" loops back to node E as "0" is no part of a "determining" sequence and that once $s3f$ is reached, all further transitions loop back to the same state. This is because once this node has been reached, the string cannot be accepted, as the only way to reach this node is through the sequence "211". Also important are the transitions between different branches, as each subsequence in a branch can potentially be the starting point of a different "determining" sequence.

3 Prolog

One method to implement the model is using Prolog. Due to the nature of prolog, the model can be easily implemented using the visual representation. It is only necessary to define the transitions and acceptance states:

```
move(e, e, 0).
```

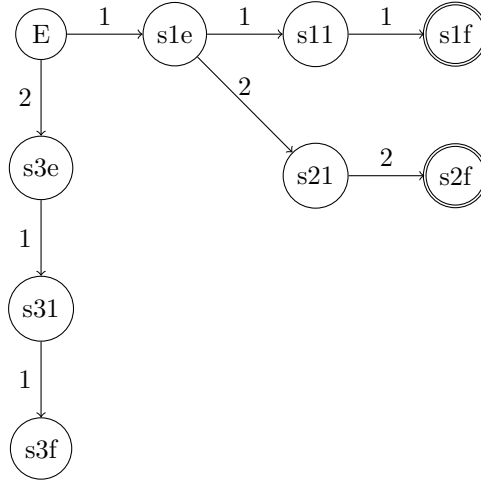


Figure 2: Depiction of the base sequence relevant for parsing any string. Note, that the accepted states can already be identified in this graph, s1f and s2f.

```

move(e, s1e, 1).
move(e, s3e, 2).

```

```

move(s1e, e, 0).
move(s1e, s11, 1).
move(s1e, s21, 2).

```

```

move(s11, e, 0).
move(s11, s1f, 1).
move(s11, s21, 2).

```

```

move(s1f, e, 0).
move(s1f, s1f, 1).
move(s1f, s21, 2).

```

```

move(s21, e, 0).
move(s21, s31, 1).
move(s21, s2f, 2).

```

```

move(s2f, e, 0).
move(s2f, s31, 1).
move(s2f, s3e, 2).

```

```

move(s3e, e, 0).
move(s3e, s31, 1).
move(s3e, s2e, 2).

```

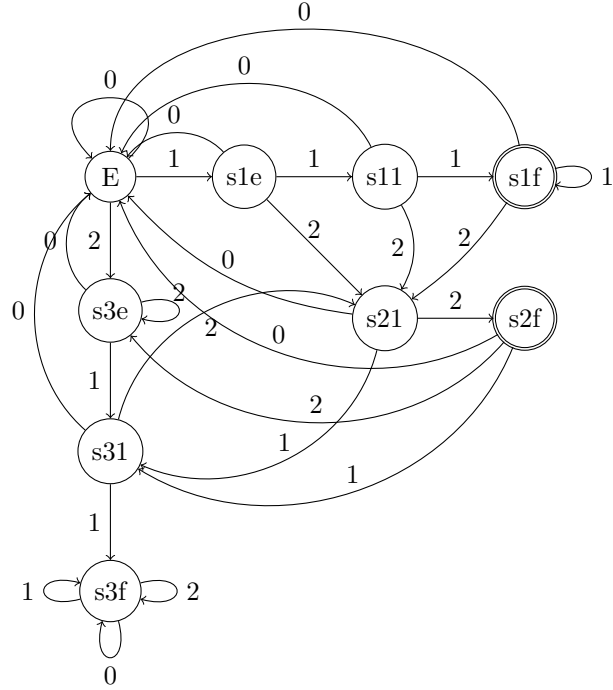


Figure 3: Depiction of the full automaton. Note that once s3f is reached, no possibility of reaching any other state exists. Further, any "0" resets any sequence and it is possible to switch between branches if the current branch could be a subsequence of another branch.

```

move(s31, e, 0).
move(s31, s3f, 1).
move(s31, s21, 2).

```

```

move(s3f, s3f, 0).
move(s3f, s3f, 1).
move(s3f, s3f, 2).

```

```

accepting_state(s1f).
accepting_state(s2f).

```

The parser provided in the lecture is used to parse the string recursively letter by letter.

```

parseDFA(InputList) :-
    parseDFAHelper(InputList, e).
parseDFAHelper([], CurrentState) :-

```

```

    accepting_state(CurrentState),
    write('Accepted'), nl;
    write('Rejected'), nl.
parseDFAHelper([Symbol|Rest], CurrentState) :-
    move(CurrentState, NextState, Symbol),
    parseDFAHelper(Rest, NextState).

```

The helper function separates out each symbol and uses the symbol to match a *move* clause definition to obtain the new state. The full script, including the test cases can be found in "dfa.pl".

If given a string of length n , the tail recursion algorithm looks at every symbol only once. Therefore, the algorithm is in $O(n)$.

4 REGEX

"REGEX", short for "regular expression" is according to [Hopcroft et al., 2007] an algebraic description of languages. They further proof that any language that can be represented using a deterministic finite automaton, can also be represented using regular expressions. Designing a regular expression is different than designing an automaton. Instead of thinking in individual states, one starts out by breaking the language down into individual patterns to then string them together. In this case we can start out with the easiest condition: The string has to end in either "111" or "122". In regex this can be denoted as

$$(111|122)\$$$

Here the "\$" enforces the string to end in the prior pattern. Before this pattern, any pattern that doesn't contain "211" can be used. This case can be broken down into 6 different patterns that can be repeated infinitely to build any valid pattern using *. Combined with the previous pattern and the addition of a special case, the regular expression results in:

$$^(0|1|20|22|212|210)*(2122|111|122)\$.$$

Note the special ending sequence "2122". As the first part can have any number of 1s and the sequence can end in "111", adding a single "2" in the first part, would allow for the acceptance of invalid patterns. The only case where "21" would not raise a potential concern is when the string ends in "122". Thus the ending sequence "2122" has to be matched separately. The "^"-symbol forces the beginning of string to also start with any of those patterns, forcing REGEX to consider the entire string for matching and not just parts.

5 Tests

To test both implementations, test cases have been derived to test various situations. The cases have been divided into two categories: Strings that should be accepted and string that should be rejected.

5.1 Accepted

Here the cases 1. and 2. validate the base cases, where only the ending sequence occurs. Cases 3., 4. and 6. test more complex cases, with longer strings. Cases 5. and 7. are specifically designed to test switching inbetween sequences, for example in 5., although the ending sequence "111" fully appears, the actual ending sequence "122" has to be identified.

1. 111
2. 122
3. 1122
4. 00120122
5. 111111122
6. 01201201111
7. 112122

5.2 Rejected

The rejection cases test either the end sequence such as cases 1. and 4. or the occurrence of "211" which is tested in all other cases. Case 3. shows an interesting problem for REGEX, as the sequence "2122" needs to be accepted, but the sequence "2111" needs to be rejected.

1. 1112
2. 211
3. 01201202111
4. 00121222
5. 001211222
6. 11112110111
7. 2110102001221200122

While the tests don't enumerate every single possible case, they test every switch of sequence/ branch and some combinations of those. These tests and parts of tests can be used to construct any other string.

6 Comparison and Learnings

Both methods can be used to solve the task of lexical analysis. But both trade interpretability for implementation complexity. Using Prolog and deterministic finite automata leads to easy interpretability by humans, as describing the process in a graph allows for easy understanding. But this method becomes very inconvenient for increasingly complex cases, as more connections have to be drawn and denoted as the alphabet or specifications increase. Here, REGEX is more scalable, enabling the notation of very complex cases in a single line of algebraic description. But this comes at the cost of interpretability, as those descriptions become hard to read and understand the more complex they get, giving more space for implementation error.

References

- [Frank Stakano, 2024] Frank Stakano, A. P. (2024). Formal languages and automata.
- [Geeks-For-Geeks, 2026] Geeks-For-Geeks (2026). Designing finite automata from regular expression (set 1).
- [Hopcroft et al., 2007] Hopcroft, J. E., Motwani, R., and Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation, 3rd edition*. Pearson Education, Inc.